School of Future Tech

Case Study Report

Matrix Addition System (C++)

Your Case Study Title

by

Sejal Singh

150096725045

PPT Link:

https://docs.google.com/presentation/d/10Ke58W1kiOQpwQ9nn_wgHNLDQtJMVDde0p8byngFxRA/edit?usp=sharing

Github Repository Link:

https://github.com/sejaltech/Final-Project-of-C-.git

1. Introduction to the Case Study

Matrices are important in mathematics and computer science. They are used in engineering, graphics, physics, and data processing.

This case study focuses on designing and implementing a **Matrix Addition System using C++**. The system allows users to input two matrices of the same size and calculates their sum.

The project demonstrates how two-dimensional arrays work in C++, how nested loops operate, and how mathematical operations can be implemented programmatically.

2. Problem Statement / Case Background (Abstract)

Background

Matrix addition is a basic mathematical operation. It is performed by adding corresponding elements of two matrices having the same dimensions.

In real-world applications, matrix operations are used in:

- Image processing
- Game development
- Scientific calculations
- Machine learning

Abstract

This case study presents the design and implementation of a **Matrix Addition System in C++**.

The program:

- Accepts user input for matrix size (rows and columns).
- Takes input for two matrices.
- Performs matrix addition.
- Displays the result in a structured format with borders.

The system also includes:

- Input validation
- Menu-driven interface
- Proper formatting using `iomanip`

This ensures correctness, clarity, and user-friendly interaction.

3. Case Study Design

The Matrix Addition System is designed as a modular program with the following components:

1️⃣User Input Module

- Accept matrix size (1–10 rows and columns).
- Accept elements for Matrix A and Matrix B.
- Validate input using a safe integer function.

2️⃣Processing Module

- Use nested loops.

- Add corresponding elements:

  `result[i][j] = A[i][j] + B[i][j];`

3️⃣ Output Module

- Display matrices in formatted bordered structure.
- Show Matrix A
- Show Matrix B
- Show Result Matrix

4️⃣ Menu System

- Option 1: Add Matrices
- Option 2: Exit

The program continues until the user chooses Exit.

  ○

4. Methods & Algorithms Technology Applied in the Problem Statement

Programming Language Used

- C++

Concepts Used

🔹 Two-Dimensional Arrays

Used to store matrix values:

```
int A[MAX][MAX];
```

🔹 Nested Loops

Used for:

- Taking input
- Performing addition
- Displaying matrices

🔹 Input Validation

The function `getValidInt()` ensures:

- User enters valid numbers
- Value remains within allowed range

🔹 Matrix Addition Algorithm

Step 1: Input matrix dimensions
Step 2: Input elements of Matrix A
Step 3: Input elements of Matrix B
Step 4: For each element:

```
For i = 0 to rows
  For j = 0 to cols
    result[i][j] = A[i][j] + B[i][j]
```

Step 5: Display result

🔹 Formatting

`iomanip` library is used for alignment (`setw()`).

-

5. Problem Statement / Case Study Implementation Details and Snapshots.

Program Structure

The program contains:

- `getValidInt()` → Safe input function
- `displayMatrix()` → Displays matrix with borders
- `printHeader()` → Displays section titles
- `printDivider()` → Prints line separator
- `main()` → Controls full execution

-

Snapshots:-

```
24    // Display matrix with borders
25    void displayMatrix(int mat[][MAX], int rows, int cols) {
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
cd "/Users/sejalsingh/Desktop/c++ final project/" && g++ matrix.cpp -o matrix && "/Users/sejals
sejalsingh@sejals-MacBook-Air c++ final project % cd "/Users/sejalsingh/Desktop/c++ final proje
ct/"matrix

╔═══════════════════════════════╗
    MATRIX ADDITION SYSTEM
        Case Study 167
╚═══════════════════════════════╝

╔═══════════════════════════════╗
          M A I N  M E N U
╠═══════════════════════════════╣
   1.  Add Matrices
   2.  Exit
╚═══════════════════════════════╝

Enter choice (1-2):
```

```
Enter choice (1-2): 1

Enter Matrix Size
──────────────────────────────────────
Rows    (1-10) : 2
Columns (1-10) : 2

╔═══════════════════════════════╗
  Enter Matrix A
╚═══════════════════════════════╝

A[1][1] = 2
A[1][2] = 2
A[2][1] = 3
A[2][2] = 3

╔═══════════════════════════════╗
  Enter Matrix B
╚═══════════════════════════════╝

B[1][1] = 3
B[1][2] = 2
B[2][1] = 3
B[2][2] = 2
```

```
R E S U L T S

Matrix A :

| 2 | 2 |
| 3 | 3 |

[ + ]

Matrix B :

| 3 | 2 |
| 3 | 2 |

[ = ]

Resultant Matrix (A + B) :

| 5 | 4 |
| 6 | 5 |

Addition Completed
```

```
Addition Completed

M A I N  M E N U

1.  Add Matrices
2.  Exit

Enter choice (1-2): 2

Goodbye! See you next time.

sejalsingh@sejals-MacBook-Air c++ final project %
```

Features of the System

✔ Menu-driven interface
✔ Input validation
✔ Clean formatted output
✔ Supports up to 10×10 matrices
✔ Uses modular functions

6. Problem Statement / Case Study Results and Conclusion.

Results

The Matrix Addition System successfully:

- Accepts valid matrix inputs.
- Performs accurate matrix addition.
- Displays output in a structured and readable format.
- Prevents invalid input using validation logic.

Conclusion

This case study demonstrates practical implementation of:

- 2D arrays
- Nested loops
- Functions
- Menu-based programs
- Input validation

The project provides a strong foundation for understanding matrix operations in C++.

It can be further enhanced by adding:

- Matrix subtraction
- Matrix multiplication
- File handling
- Graphical interface

7. References

- C++ Documentation (iostream, iomanip, limits)
- Standard textbooks on Data Structures and C++
- Classroom notes and lectures
- Online C++ learning resources
- web3school